Discord Bot Maintainer — Security, Evidence Intake and Release Guide

v1.1 | Technical maintainer / organizer; production documentation release in package v3.22; not participant instructions

PRODUCTION DOCUMENTATION RELEASE (v3.22) — USER-DIRECTED GATE WAIVER; TECHNICAL TESTS REMAIN UNVERIFIED

A maintainer-only runbook for controlling sensitive endpoints, checking an exact bot build, validating OCR output, and running the target-runtime/Discord smoke sequence. No bot source is modified or repackaged; the current RC3 target package and live credentials are unavailable for validation here.

Prepared 29 October 2026 • Policy version v3.15

1. Release status and scope

⚓ **Quick hint:** A polished document is not release approval; keep blockers visible until the required evidence exists.

Documentation release status — RELEASED UNDER USER-DIRECTED GATE WAIVER. The latest RC3 source package was not available in the accessible workspace. The old archive passed 32/32 unit tests under Linux Node v24.22.1, but its reported-screenshot exact Damage test failed by 60 (expected 87373, extracted 87313) and its four-image test timed out at 120 seconds. These are historical results, not RC3 results; the bot runtime remains unverified and is not certified by this documentation release.

Project constraint: keep the OCR workflow local; retain OpenCV and PaddleOCR as planned/local components and benchmark any engine choice against frozen fixtures. Never silently replace an OCR engine solely because it is convenient. Ambiguous critical values remain in human review.

2. Security and credential response

⚓ **Quick hint:** Use only the authorized tool and permissions; retain a manual fallback and never expose credentials.

1. Do not use archived score_report_imgToDiscord.html for live participant data; previous review found a browser-side webhook/posting path. Do not distribute it as a safe live reader.
2. Do not paste webhook URLs, tokens, bot credentials, private configuration, participant records, or live screenshots into public documentation, browser source, repository history, or a public channel.

3. An operator with access to the credential issuer must establish whether the exposed webhook is still live, revoke or regenerate it if applicable, and record the issuer confirmation, date/time, credential identifier (never the secret), and responsible operator. This guide cannot revoke it on its own.
4. Remove browser-side posting from any replacement and use a server-side/approved secure send path only if the actual replacement architecture supports it. Require explicit participant review before any submission is sent.
5. Scan the exact release candidate and every archive/HTML asset for webhook patterns, token-like strings, .env/config secrets, source maps, logs and test fixtures. Document all findings. A clean documentation package does not prove that the actual code bundle is clean.

3. Build identity and local runtime

⚓ **Quick hint:** Check the stated authority, required evidence, and completion criteria before acting; do not infer an approval that is not recorded.

6. Obtain the actual RC3 package described by plan1.txt. Record exact filename, SHA-256, repository commit/build ID, package manager lockfile, Node version, OS build, OCR engine/model versions and relevant OpenCV dependencies.
7. Run tests only from a verified copy of that source. Preserve the original package untouched. Use local OCR assets and explicitly disable outbound/OCR API services for the required local workflow.
8. Review dependency install scripts and runtime configuration before execution; do not inject production tokens during unit tests. Use an approved test account and non-production channel for smoke tests.
9. Verify that PaddleOCR/OpenCV remain local components and that Tesseract.js comparisons use frozen fixture images. Any optional engine must be behind the planned feature flag and evidence-based selection path; conflicting candidates remain unresolved for human review.

4. Required acceptance sequence

⚓ **Quick hint:** Check the stated authority, required evidence, and completion criteria before acting; do not infer an approval that is not recorded.

Gate	Run / inspect	Required evidence	Current status
Source identity	Fingerprint latest RC3 source and environment.	Filename, SHA-256, commit/build ID, Node/npm versions.	OPEN — source unavailable
Unit / regression tests	Run full unit suite and specific failed screenshot case.	Complete logs; no hidden skips; exact field-level assertions.	Older archive 32/32 only; not RC3
OCR numeric accuracy	Run required screenshot; frozen crop fixtures; one- and four-image cases.	Expected vs extracted per field, confidence/candidate logs, duration/timeout.	OLD archive failed Damage by 60; four-image timed out
OpenCV crop isolation	Test independently isolated candidate crops for Damage and other numeric fields.	Geometry/bounds, crop image, each engine candidate, decision rationale.	OPEN for exact RC3
Windows/Node 24	Run the documented full target runtime with native dependencies.	Windows logs, package/runtime versions, exit codes, result report.	OPEN

Native Tesseract comparison	Run same frozen local fixtures against required comparison engine.	Per-field exact-match and complete-record accuracy.	OPEN
Discord authentication/permissions	Authorized live login, channel/role permissions and least privilege.	Test account, permission matrix, timestamped successful/denied cases.	OPEN
Attachment/rate-limit/ recovery	Upload handling, large/invalid files, API throttling, restart and recovery.	Logs, retries/backoff, no duplicate accepted results, clean restart.	OPEN
Human correction/fallback	Ambiguous OCR, cancel/retry, manual route.	Original screenshot + OCR candidate + accepted correction/audit trail.	OPEN
Security release scan	Inspect exact release source and all package assets.	Scan report, disposition of each finding, credential-rotation evidence.	OPEN

5. Operational handling of evidence

⚓ **Quick hint:** Preserve the original evidence and distinguish an observed fact from an OCR, bot, or calculator interpretation.

1. Treat OCR output as a candidate. Compare the original screenshot for player identity, Damage, battle result, map, mode and time; only include other fields when the actual product requires them.

2. If candidates conflict, do not select the value that best fits a score. Preserve the evidence and ask for human review or a clearer original.

3. Keep the original image, extracted values, alternative candidates, confidence/decision, participant correction, accepted metrics, the approved scoring calculation, the match-outcome calculation, verified bracket decision, and any individual award as distinct but linked traceable records. The approved performance-based calculation determines the outcome; preserve the trail showing how accepted metrics produced it.

4. Prevent duplicate posts and silent overwrite. Use a stable event/game/participant identifier; corrections create audit records rather than erase the previous value.

5. If the bot is down, use the published manual fallback. Bot availability never authorizes a participant to bypass the official evidence process.

Governing rule for all bot commands, responses, notices, and staff operations: any command or action that publishes or changes bracket progression must use the verified match outcome calculated from accepted individual-performance metrics (including Damage and Kills) under the approved scoring method and published aggregation, rounding, and tie-break rules. The bot must not treat a raw statistic, a standalone dblns value, a manually asserted result, or an unverified calculator display as a substitute for that calculation. If evidence or formula parity is unresolved, the command/action must hold the affected score or bracket change, preserve traceable inputs and outputs, and route the issue to an authorized human decision-maker. This is a governing documentation requirement; the actual bot implementation remains unverified.

6. Current security disposition

⚓ **Quick hint:** Use only the authorized tool and permissions; retain a manual fallback and never expose credentials.

Not closed — Credential revocation/rotation in the issuing service has not been verified. Actual RC3 source secret scanning, browser-side posting removal, Windows OCR acceptance and live Discord smoke tests have not been run on the exact target build. The release cannot pass the security/deployment gate on documentation alone.

7. Definition of done

⚓ **Quick hint:** Use the defined term consistently; if two documents differ, follow the explicitly named governing source.

Bot/security gate passes when — The exact RC3 build passes the full Windows Node 24 suite, screenshot/field accuracy requirements, native Tesseract comparison, OpenCV crop tests, live Discord auth/permissions/attachment/rate-limit/restart tests, human correction and fallback checks, source/asset secret scan, credential rotation/removal proof, and an authorized maintainer review. All logs identify the tested build.

Project source note

⚓ **Quick hint:** Historical material provides context only; do not present it as current validation or approval.

The plan1.txt roadmap and addedImgProcess.txt establish the local-first OCR constraints and target tests. This is technical-maintainer documentation only; it is not user-facing bot source and makes no claim that an untested build is production-ready.